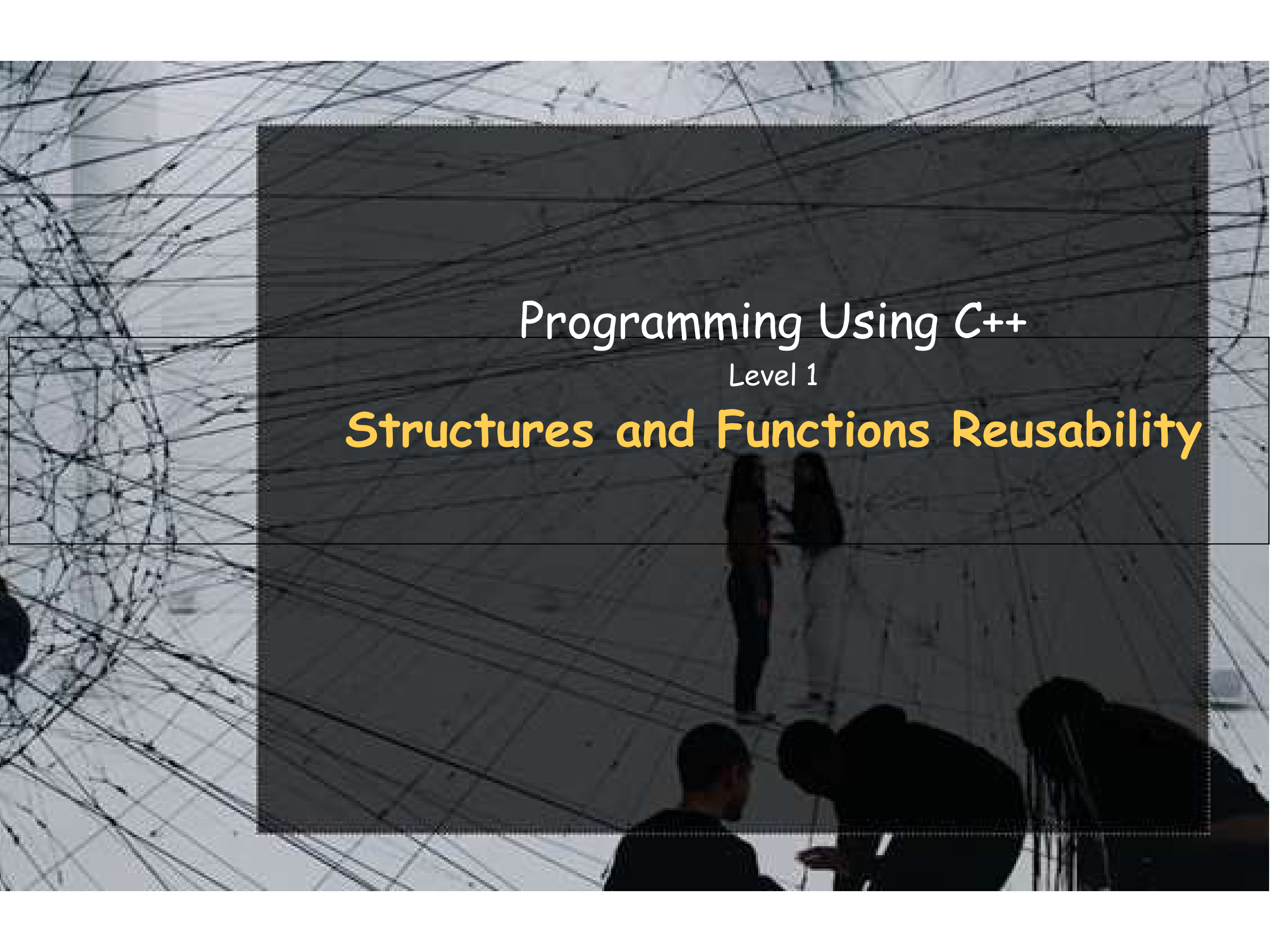


Programming Using C++ Level 1

Structures and Functions Reusability



Programming Using C++

Level 1

Structures and Functions Reusability

Definition

- ◆ A structure can group related data into one custom data type.
- ◆ Functions can receive a structure as a parameter and work with all of its members.
- ◆ Structure reusability means the same structure and functions can be reused for multiple objects containing the same type of information.

Explanation

- ◆ This program defines a `strInfo` structure containing personal information.
 - ▣ Name, Age, City, Country
 - ▣ MonthlyIncome, YearlyIncome
 - ▣ Gender, Married
- ◆ `ReadInfo()` receives the structure by reference so it can store user input directly into the original structure.
- ◆ `PrintInfo()` receives the structure by const reference so it can read and display the information without modifying it.
- ◆ The same functions can be reused with other `strInfo` variables.

Steps in main()

1 Define the structure

◆ Create strInfo and add all required personal information members.

2 Create the ReadInfo() function

◆ Pass the structure by reference and store the user's input in its members.

3 Calculate yearly income

◆ Calculate YearlyIncome using $\text{MonthlyIncome} \times 12$.

4 Create the PrintInfo() function

◆ Pass the structure by const reference and display its information.

5 Create a structure variable

◆ Create Person1Info using the strInfo structure.

6 Read the information

◆ Pass Person1Info to ReadInfo().

7 Print the information

◆ Pass Person1Info to PrintInfo().

💡 Example

◆ Structure variable:

```
strInfo Person1Info;
```

◆ Reading information:

```
ReadInfo(Person1Info);
```

◆ Printing information:

```
PrintInfo(Person1Info);
```

◆ The same functions can also be used with another variable:

```
strInfo Person2Info;
```

```
ReadInfo(Person2Info);
```

```
PrintInfo(Person2Info);
```

➡ Result

◆ The program reads the user's personal information, calculates yearly income, and then displays all stored information.

◆ For example, if MonthlyIncome is 3000:

YearlyIncome = $3000 \times 12 = 36000$

Note

- ◆ `ReadInfo(strInfo& Info)` uses a non-const reference because it needs to modify `Info`.
- ◆ `PrintInfo(const strInfo& Info)` uses a const reference because it only reads `Info`.
- ◆ The dot operator `.` is used to access structure members, such as `Info.Name` and `Info.Age`.
- ◆ The ternary operator is used to display the Boolean `Marrid` value as `Yes` or `No`.

⊘ Warning

◆ If a function only needs to read a structure, using const reference is safer because it prevents accidental modification.



Comparison

▨ Passing Structure By Reference

- ✓ Allows the function to modify the original structure.
- ✓ Used by ReadInfo() to store user input.
- ✓ Avoids creating a separate copy of the structure.

▤ Passing Structure By Const Reference

- ✓ Allows the function to read the original structure.
- ✓ Prevents the function from modifying it.
- ✓ Used by PrintInfo() because it only needs to display the data.

✦✦ Summary

- ✓ Structures group related information into one custom data type.
- ✓ Functions can receive structures by reference or const reference.
- ✓ Combining structures and functions makes code more reusable and organized.

Thank you 😊